

BÀI THỰC HÀNH DECORATOR PATTERN

- ✓ Backend: ASP.NET Core Web API
 - ✓ Frontend: React
 - ✓ Áp dụng Decorator Pattern chuẩn GoF
 - ✓ Thêm/bớt tính năng động (dynamic behavior) cho đối tượng
-

BÀI TOÁN

Hệ thống E-commerce cho phép người dùng đặt đồ uống (Coffee).

Người dùng có thể **thêm topping** như:

- Milk
- Sugar
- Ice

 Yêu cầu:

- Không sửa class gốc
 - Có thể thêm topping linh hoạt runtime
-

KIẾN TRÚC HỆ THỐNG

```
Frontend (React)
  ↓
POST /api/orders/coffee
  ↓
CoffeeService
  ↓
Base Coffee (SimpleCoffee)
  ↓
Decorator (MilkDecorator, SugarDecorator, IceDecorator)
  ↓
ICoffee
```

BACKEND – ASP.NET Core Web API

Tạo project

```
dotnet new webapi -n DecoratorDemo
```

```
cd DecoratorDemo
```

Cấu trúc thư mục

```
DecoratorDemo
├── Controllers
│   └── CoffeeController.cs
├── Services
│   └── CoffeeService.cs
├── Decorators
│   ├── CoffeeDecorator.cs
│   ├── MilkDecorator.cs
│   ├── SugarDecorator.cs
│   └── IceDecorator.cs
├── Coffee
│   ├── ICoffee.cs
│   └── SimpleCoffee.cs
├── Models
│   └── CoffeeRequest.cs
└── Program.cs
```

1. COMPONENT INTERFACE

Coffee/ICoffee.cs

```
namespace DecoratorDemo.Coffee;

public interface ICoffee
{
    string GetDescription();
    double GetCost();
}
```

2. CONCRETE COMPONENT

Coffee/SimpleCoffee.cs

```
namespace DecoratorDemo.Coffee;

public class SimpleCoffee : ICoffee
{
    public string GetDescription() => "Coffee";

    public double GetCost() => 2.0;
}
```

```
}
```

3. DECORATOR BASE

Decorators/CoffeeDecorator.cs

```
using DecoratorDemo.Coffee;

namespace DecoratorDemo.Decorators;

public abstract class CoffeeDecorator : ICoffee
{
    protected ICoffee _coffee;

    public CoffeeDecorator(ICoffee coffee)
    {
        _coffee = coffee;
    }

    public virtual string GetDescription()
        => _coffee.GetDescription();

    public virtual double GetCost()
        => _coffee.GetCost();
}
```

4. CONCRETE DECORATORS

MilkDecorator

```
using DecoratorDemo.Coffee;

namespace DecoratorDemo.Decorators;

public class MilkDecorator : CoffeeDecorator
{
    public MilkDecorator(ICoffee coffee) : base(coffee) {

    }

    public override string GetDescription()
        => _coffee.GetDescription() + ", Milk";

    public override double GetCost()
        => _coffee.GetCost() + 0.5;
}
```

SugarDecorator

```
using DecoratorDemo.Coffee;

namespace DecoratorDemo.Decorators;

public class SugarDecorator : CoffeeDecorator
{
    public SugarDecorator(ICoffee coffee) : base(coffee)
    { }

    public override string GetDescription()
        => _coffee.GetDescription() + ", Sugar";

    public override double GetCost()
        => _coffee.GetCost() + 0.3;
}
```

IceDecorator

```
using DecoratorDemo.Coffee;

namespace DecoratorDemo.Decorators;

public class IceDecorator : CoffeeDecorator
{
    public IceDecorator(ICoffee coffee) : base(coffee) { }

    public override string GetDescription()
        => _coffee.GetDescription() + ", Ice";

    public override double GetCost()
        => _coffee.GetCost() + 0.2;
}
```

5. REQUEST MODEL

Models/CoffeeRequest.cs

```
namespace DecoratorDemo.Models;

public class CoffeeRequest
```

```
{  
    public List<string> Toppings { get; set; }  
}
```

6. SERVICE

Services/CoffeeService.cs

```
using DecoratorDemo.Coffee;  
using DecoratorDemo.Decorators;  
  
namespace DecoratorDemo.Services;  
  
public class CoffeeService  
{  
    public ICoffee CreateCoffee(List<string> toppings)  
    {  
        ICoffee coffee = new SimpleCoffee();  
  
        foreach (var topping in toppings)  
        {  
            switch (topping.ToLower())  
            {  
                case "milk":  
                    coffee = new MilkDecorator(coffee);  
                    break;  
                case "sugar":  
                    coffee = new SugarDecorator(coffee);  
                    break;  
                case "ice":  
                    coffee = new IceDecorator(coffee);  
                    break;  
            }  
        }  
  
        return coffee;  
    }  
}
```

7. CONTROLLER

Controllers/CoffeeController.cs

```
using Microsoft.AspNetCore.Mvc;  
using DecoratorDemo.Models;
```

```

using DecoratorDemo.Services;

namespace DecoratorDemo.Controllers;

[ApiController]
[Route("api/coffee")]
public class CoffeeController : ControllerBase
{
    private readonly CoffeeService _service;

    public CoffeeController(CoffeeService service)
    {
        _service = service;
    }

    [HttpPost]
    public IActionResult CreateCoffee(CoffeeRequest request)
    {
        var coffee =
            _service.CreateCoffee(request.Toppings);

        return Ok(new
        {
            description = coffee.GetDescription(),
            cost = coffee.GetCost()
        });
    }
}

```

✿ 8. ĐĂNG KÝ DI

Program.cs

```

using DecoratorDemo.Services;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllers();
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

builder.Services.AddTransient<CoffeeService>();

var app = builder.Build();

```

```
if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}

app.UseHttpsRedirection();
app.MapControllers();
app.Run();
```

TEST API

POST /api/coffee

```
{
  "toppings": ["milk", "sugar"]
}
```

 Response:

```
{
  "description": "Coffee, Milk, Sugar",
  "cost": 2.8
}
```

FRONTEND – REACT

Tạo project

```
npx create-react-app coffee-ui
cd coffee-ui
npm install axios
```

src/App.js

```
import React, { useState } from "react";
import axios from "axios";

function App() {
  const [toppings, setToppings] = useState([]);
  const [result, setResult] = useState("");

  const toggle = (item) => {
    setToppings((prev) =>
```

```

        prev.includes(item)
        ? prev.filter((x) => x !== item)
        : [...prev, item]
    );
};

const order = async () => {
    const res = await
    axios.post("https://localhost:xxxx/api/coffee", {
        toppings: toppings,
    });

    setResult(`${res.data.description} -
    ${res.data.cost}`);
};

return (
    <div style={{ padding: 40 }}>
        <h2>Order Coffee</h2>

        <label><input type="checkbox" onChange={() =>
toggle("milk")} /> Milk</label><br/>
        <label><input type="checkbox" onChange={() =>
toggle("sugar")} /> Sugar</label><br/>
        <label><input type="checkbox" onChange={() =>
toggle("ice")} /> Ice</label><br/>

        <br/>
        <button onClick={order}>Order</button>

        <h3>{result}</h3>
    </div>
);
}

export default App;

```

LUỒNG XỬ LÝ

Khi user chọn Milk + Sugar

```

SimpleCoffee
  ↓
MilkDecorator
  ↓
SugarDecorator

```


👉 Kết quả:

```
Coffee, Milk, Sugar  
Cost = 2.0 + 0.5 + 0.3
```

🔗 MỤC TIÊU HỌC ĐƯỢC

Sinh viên hiểu rõ:

- ✓ Decorator Pattern là gì
 - ✓ Khác với kế thừa (Inheritance)
 - ✓ Thêm behavior động (runtime)
 - ✓ Áp dụng thực tế trong hệ thống lớn
-

🚀 BÀI TẬP MỞ RỘNG

1. Thêm topping:
 - Chocolate
 - Caramel
2. Thêm Size:
 - Small / Medium / Large (Decorator nâng cao)
3. Refactor:
 - Dùng Dictionary thay switch-case
4. Nâng cấp:
 - Kết hợp với Factory Pattern (tạo base coffee)